**PARTE 2 - Ejercicios de seudocódigo**
**Duración: 60 minutos**

# EJERCICIO 1

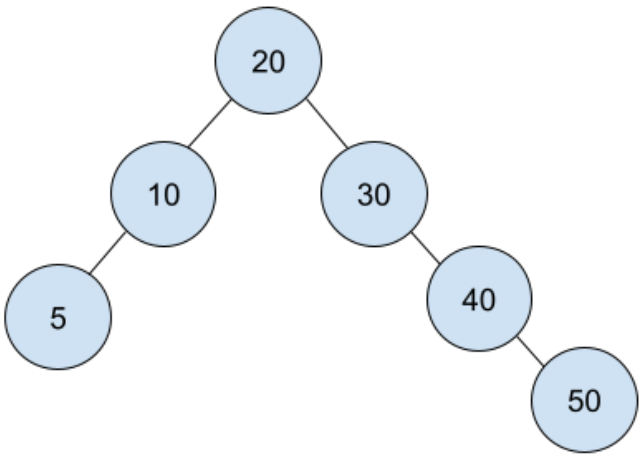
## CONSIDERACIONES IMPORTANTES

- **Se debe mostrar paso a paso la inserción de cada clave.**
- **Cuando se produce un desbalance, se debe identificar el nodo desbalanceado, el tipo de desbalance y los distintos nodos que se ven involucrados en la operación de balanceo (k1, k2, k3).**
- **En caso de rotaciones complejas, se puede obviar mostrar el estado intermedio luego de realizar la primera rotación.**

A continuación, se muestra un árbol binario de búsqueda, resultado de haber insertado algunas claves. Se desea insertar las claves 45, 35, 25 y 37 de tal forma que el árbol resultante en cada inserción sea AVL.

En cada paso previo o posterior a la inserción se debe:

- Dibujar el árbol resultante.
- Señalar si ocurre un desbalance y en qué nodo.
- Indicar la rotación necesaria, si corresponde, y mostrar el árbol resultante.



# EJERCICIO 2

## CONSIDERACIONES IMPORTANTES

- **Deben desarrollarse en detalle todos los métodos que se invoquen.**
- **Se tendrá en cuenta la eficiencia de los algoritmos desarrollados, calificando mejor aquellos que tengan mejor tiempo de ejecución o mejor performance general.**
- **No es necesario desarrollar los métodos de los TDAs utilizados si estos corresponden a las variantes vistas en clase.**
- **Se debe justificar claramente la elección del TDA utilizado para organizar la información, explicando por qué es el más adecuado para este problema.**
- **Se deben cumplir todos los pasos estándar de desarrollo de algoritmos, Lenguaje Natural, Pre y Post Condiciones, Pseudocódigo y Análisis del Orden del Tiempo de Ejecución.**

## Recomendador de Películas

**Escenario**: Una plataforma de streaming quiere organizar su catálogo de películas de manera que las búsquedas sean lo más eficientes posible. Para ello, contrata a su empresa unipersonal. Su tarea consiste en analizar las distintas estructuras de datos vistas durante la cursada y seleccionar la más adecuada para resolver este problema de forma eficiente.

Cada película del catálogo está caracterizada por: Año de estreno (entero), Título (cadena de caracteres), Género (cadena de caracteres) y un Score (real, entre 1 y 10).

**Ejemplo:**

Año; Título ; Género ; Score
2009 ; Avatar ; Sci-Fi ; 8.0
1960 ; Psicosis ; Terror ; 5.6
1998 ; Blade ; Acción ; 7.1
1975 ; Tiburón ; Suspenso ; 7.1
1942 ; Casablanca ; Romance ; 8.5
2014 ; Interestelar ; Sci-Fi ; 8.6
2000 ; Gladiator ; Acción ; 8.5

**Objetivos**

Se solicita implementar un sistema que permita:

- Construir un TDA a partir de las películas listadas en el archivo Pelis.txt.
- Recorrer la estructura utilizando el recorrido adecuado para cumplir con el objetivo.
- Filtrar las películas recomendadas (aquellas con score > 7.5) y devolver la lista, además escriba la lista de recomendación en un archivo llamado Recomendadas.txt.

Nota: la lista de películas recomendadas debe respetar el orden alfabético de los títulos.

EJERCICIO 2 – ALTERNATIVO (MÁS DIFÍCIL)

## Escenario

Una plataforma de streaming desea mejorar la organización y búsqueda de su catálogo de películas para ofrecer una experiencia más eficiente a sus usuarios. Para ello, contrata a su empresa unipersonal. Su tarea consiste en analizar las distintas estructuras de datos vistas durante la cursada y seleccionar la más adecuada para resolver este problema de forma eficiente.

El objetivo del sistema es **proporcionar un catálogo que permita realizar búsquedas rápidas y eficientes basadas en diferentes criterios, ofreciendo resultados de las más viejas a las más nuevas.** Además, el catálogo se actualiza con nuevas películas todas las semanas, por lo que la estructura elegida debe soportar inserciones frecuentes de forma que mantenga el orden y la eficiencia en las búsquedas.

Cada película del catálogo está caracterizada por los siguientes atributos:

- Año de estreno (entero)
- Título (cadena de caracteres)
- Género (cadena de caracteres)
- Score (real, entre 1 y 10)

## Objetivos

Se solicita implementar un sistema que permita:

- El sistema debe ser capaz de **leer un archivo de texto (Pelis.txt) con la lista de películas y construir una estructura de datos eficiente para almacenarlas, permitiendo búsquedas rápidas por score y nombre.**
- Cada línea del archivo tiene el formato:
  - Año; Título; Género; Score.
  - Ejemplo:
    - 2009 ; Avatar ; Sci-Fi ; 8.0
    - 1960 ; Psicosis ; Terror ; 5.6
    - 1998 ; Blade ; Acción ; 7.1
- Implementar una f**unción que permita obtener todas las películas cuyo score esté dentro de un rango especificado por un mínimo y un máximo**.
  - Si no se especifica un mínimo, se deben incluir todas las películas con score menor o igual al máximo.
  - Si no se especifica un máximo, se deben incluir todas las películas con score mayor o igual al mínimo.
  - Si no se especifican ni mínimo ni máximo, se deben devolver todas las películas.
  - Los resultados deben estar ordenados por el año de estreno de las películas en orden ascendente
- Implementar una **función que permita obtener todas las películas de un género específico.**
  - Los resultados deben estar ordenados por el año de estreno de las películas en orden ascendente.